

Characterization of Operational Failures from a Business Data Processing SaaS Platform

Catello Di Martino,
Zbigniew Kalbarczyk,
Ravishankar K. Iyer
Coordinated Science Laboratory
University of Illinois at Urbana Champaign
Urbana, IL, USA
{dimart,kalbarcz,rkiyer}@illinois.edu

Geetika Goel,
Santonu Sarkar,
Rajeshwari Ganesan
Next Gen Computing Lab
Infosys Labs, Bangalore, India
{geetika_goel,santonu_sarkar01,rajeshwari_ganesan}
@infosys.com

ABSTRACT

This paper characterizes operational failures of a production Custom Package Good Software-as-a-Service (SaaS) platform. Events log collected over 283 days of in-field operation are used to characterize platform failures. The characterization is performed by estimating (i) common failure types of the platform, (ii) key factors impacting platform failures, (iii) failure rate, and (iv) how user workload (files submitted for processing) impacts on the failure rate. The major findings are: (i) 34.1% of failures are caused by unexpected values in customers' data, (ii) nearly 33% of the failures are because of timeout, and (iii) the failure rate increases if the workload intensity (transactions/second) increases, while there is no statistical evidence of being influenced by the workload volume (size of users' data). Finally, the paper presents the lessons learned and how the findings and the implemented analysis tool allow platform developers to improve platform code, system settings and customer management.

Categories and Subject Descriptors

D.2.4 [Software Engineering]: Software/Program Verification—*Reliability, Statistical methods*

General Terms

Measurement, Reliability

Keywords

Cloud Computing, SaaS, Failure Analysis, Logs, Robustness

1. INTRODUCTION

In Software-as-a-Service (SaaS) paradigm, the software is hosted remotely, typically on private or public cloud infrastructures. Customers access provided applications using

thin clients such as web browsers. Data-processing applications, such as CRM (Customer Relationship Management) and CPG (Custom Package Good) represent an excellent example of SaaS applications. Such systems process very large amount of business-oriented data on a daily basis. In 2012, 40% of CRM systems sold were cloud/SaaS-based, CPG are the second biggest spenders on CRM with a year growth of 12.5% over 2013 [1, 10]. As a consequence, the volume of data destined to SaaS platforms is expected to grow in the years to come.

Despite CRM and CPG SaaS products represent an interesting business model for the elimination of upfront investment to software and infrastructures, little is known about the failures that such platforms can experience and what causes them. Erratic behavior can in fact become a barrier to the future development of SaaS and jeopardize both customers and the service providers that rely on such platforms for their businesses.

This paper contributes to the body of empirical knowledge on operational failures of a production CPG SaaS platform. The platform processes large volume of data files, received from 42 different customers scattered across 24 countries around the world. The data consist of files containing daily, weekly and monthly sales, inventory, or order details. On average, 3000 files/92 GB are received and processed each day. Operations consist of massive batch transformation (e.g., cleansing and harmonization) and processing of raw business data in uploaded files. Processing results are fed to several downstream business intelligence applications. The platform adopts state-of-the-art virtualization technology in a dedicated data center and runs over several virtual machines executing Microsoft Windows Server 2008, .NET platform, and SQL Server over VM-ware products.

In this paper, we focus on failures of the SaaS platform that occur during the processing of customers' data. They can have a number of causes, including software bugs, performance issues and unexpected values present in customers' data. Failures are characterized from a rigorous analysis of the data extracted from the platform and by the operating system event logs collected over 283 days of in-field operation. Results address the following research questions: i) what problems cause operational failures, and ii) what factors influence the failure rate of the studied platform?

Overall, this paper provides one of the few available studies analyzing the operational failures of a real production SaaS platform and the factors impacting it. Our major

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

Copyright is held by the author/owner(s). Publication rights licensed to ACM.

ICSE Companion'14, May 31 – June 7, 2014, Hyderabad, India
ACM 978-1-4503-2768-8/14/05
<http://dx.doi.org/10.1145/2591062.2591172>

Table 1: Major findings and their implications.

Findings	Implications
(1) 34.1% of platform failures are caused by unexpected values in the customer data.	The heterogeneity of customers' data makes it hard to design effective data validation procedures. Understanding of failure data helps to increase the reliability of a platform by guiding i) the design of effective exception handling and ii) the creation of robustness assessment campaigns (e.g., fault injection).
(2) Failures because of timeout (32.7%) are common in components of the platform that make heavy use of the database, and they tend to occur in rapid succession.	Shared resources like the DB can become the source of timeout errors due to high levels of concurrent data-processing. Timeout errors can propagate and cause other components to fail.
(3) 73.1% of all failures belong to only 3 specific failure types (out of a total of 42), namely timeout errors (32.7%), package/execution failures (failure in invoking/executing of external packages; 10.6%), and unknown (empty log records; 29.8%).	The majority of failures consist in a small number of reappearing failure types. Eliminating such problems can substantially improve the reliability of the platform. Fault injection can be used to pinpoint the weakness of such components and suggest reliability improvements.
(4) 95% of the platform failures occur in five platform stages (out of a total of 18), accounting for 37% of the total LOC.	Analysis of field failures can improve the allocation of the testing efforts. Focusing the testing on a few components can substantially improve the reliability of the platform.
(5) 30% of failures happen within 1h of the previous failure. The failure inter-arrival data can be fitted with a Fatigue Life Distribution. After failures due to timeout errors (32%) have been removed, failure inter-arrival time is exponentially distributed.	Timeout errors accumulate over time. Eliminating timeout errors would increase the Mean Time Between Failures (MTBF) by a factor of 1.94 (from 7h 2min to 13h 41min).
(6) Changes in the platform software due to major updates have a limited impact on the platform failure rate.	The increase in the failure rate observed in the analysis is not due to new software defects in platform updates. The development process is mature and does not introduce new software defects during the observation period.
(7) Increase in the failure rate depends mainly on increase in the workload intensity (i.e., the rate at which files are submitted for processing) rather than defects in the new version of the platform software.	The level of concurrency forced in the platform by intensive workload has an impact on the failure rate. Estimation of the workload intensity can be beneficial in supporting decision whether to scale the system capacity in order to control the platform failure rate (e.g., in cloud environments).
(8) There is no statistical evidence that workload volume (i.e., size of the data included in submitted files for processing) impact the platform failure rate.	The platform can scale to handle high volumes of data without being affected.
(9) The platform failure rate (rate of file processing failures) does not change over time despite the usage of the platform grows (workload intensity). Therefore, the number of failures grows as well.	A workload model taking into account the heterogeneity of customers' data must be considered during the testing phases of data processing platform.

findings and implications are summarized in Table 1. The findings presented in this paper are of interest to a broad audience, including practitioners and academic researchers. In summary, our major findings are:

- 95% of the platform failures occur in a few software modules, accounting for 37% of the total LOC (Lines of Code). 73.1% of all failures belong to only 3 specific failure types; 34.1% of all failures are caused by unexpected values in customers' data. New customers shows higher rate of data-induced failures and can affect services delivered to other customers, for instance, due to timeout errors (32.7%).
- Increase in the intensity of the user workload (i.e., the rate at which files are submitted for processing) causes an increase in the platform failure rate, but there is no statistical evidence of correlation with the workload volume (i.e., size of the data included in submitted files for processing). Consequently, the challenge is to ensure high reliability while keeping up with the continuously increasing amount of heterogeneous files to analyze.

The analysis approach and implemented tools can be applied to a broad range of systems. Results show that an in-depth analysis of failure data is the key to understanding factors that impact platform failures. Derived insights provide valuable feedback to the platform developers allowing to identify effective improvement to the platform reliability, which include code fixes as well as improvement to system configuration, virtual machine allocation policies and customer management, described in the paper. The findings and the implemented analysis tool allowed platform developers to fix 20-40% of the data-induced failures per customer, and about 85% of failures due to timeout.

2. TARGET SYSTEM

The platform is deployed over several processing nodes (application servers) and database servers. It is composed of several elaboration stages organized in a computation pipeline. Several processing phases are carried out through a specific workflow that depends on the type of data to process (e.g., weekly sales or order details). Functionally, the platform contains 7 key processing components or services. Each of the 7 key stages has sub-stages, which total to 18 processing stages. Some of the stages can have multi-thread functionality enabled within them, and overall up to 6 parallel threads can be run in the application.

Data validation (P1): Data are periodically received from the clients. This stage checks the structure and validity of the data format (e.g., number and type of fields).

Data notification (P2): This component sends notifications to various stakeholders when errors are encountered, for instance, concerning the data format.

Data verification (P3): Received data are scanned for potential viruses and verified at the semantic level for correctness and consistency according to pre-specified formats.

Data transformation (P4): Business transformations are applied to harmonize the data.

Data commit (P5): Data are sorted and moved to their respective final repositories, where these are committed.

Data Archiving (P6): Data archives are created and stored for audit purposes.

Data distribution (O4): The processed and harmonized data are sent to downstream systems for inventory management and business intelligence.

Each stage of the platform is implemented as a set of .NET applications. All services are independent of one another and perform their functions by periodically polling for relevant files, status code changes, and DB entries. The system

Table 2: Summary of the considered logs.

LOG FILES		date from	date to	# of days	# of ENTRIES		
					INFO	WARNING/ERRORS/FAILURES	TOTAL
APP SERVER OS	APPLICATION LOGS	9/15/11	10/11/12	392	NA	17487	17487
	EXCEPTION LOGS	9/15/11	10/11/12	392	363106	27112	400593
	OS LOGS	9/15/11	10/11/12	392	34447	1723	36180
DB SERVER OS	DB LOGS	9/15/11	10/11/12	392	61092	19488	101223
PLATFORM LOGS		1/1/12	10/9/12	283	2015618	1475	2017093

integrates several Commercial Off The Shelf (COTS) components for common functionalities, like data manipulation and transformation.

2.1 Data Source

The considered data are collected in several log files summarized in Table 2. They cover a period of 283 days in which the system received 275,790 files from 42 customers in 11 different countries and include: i) **platform logs** produced by the applications under study, ii) **OS logs** collected by the guest machines hosting the core services, iii) **DB logs** produced by the staging backend database, iv) **application logs**, collected by the OS, encompassing all the errors encountered by the platform modules and captured by the OS, and v) **exception logs** that include the stack trace of the generated exceptions.

Platform logs consist of several files stored as plain text generated by specific logging procedures implemented as part of the platform code. Entries are collected as client data traverses through the processing stages and include detailed information such as the size of the customer file, the number of records, client information, time, and, in the case of a failure, the computing stages involved in the failure and logged exit code. **OS, application, and exception logs** are obtained from the event-logging service in the Windows Server 2008 operating system. The event-logger software records information on errors that occur in the various subsystems, such as memory, disk, and network subsystems, as well as information on other system events, such as reboots and shutdowns.

Two types of failures are present in the considered data. The first, referred to as *validation failures*, are due to errors caused by user mistakes (e.g., a wrong/corrupted client file) and are detected during the first stage (P1) of the computation (data validation stages). Files failing the validation stage are rejected by the platform. The second, referred to as *platform failures*, can manifest in any stage of the computation, and are caused by system-related issues, including residual bugs, operating system errors, and hardware/network problems. The impact of an error can range from a minor problem to a platform hang/crash or database corruption. In this study, we focus on the analysis of platform failures because of their higher impact.

3. FAILURE ANALYSIS WORKFLOW

The workflow to analyze the logs is implemented as a set of Perl scripts and SAS procedures organized in a modular analysis package that forms the foundation of an automated analysis tool. The analysis consists of three main steps.

Step 1: Data collection, filtering, and event extraction. Initially, the data are collected from the platform and the operating systems (application server and database server) in the form of logs. Collected data are filtered in order to leave only events of interest, i.e., warning, and error or failure events. This phase encompasses several data

manipulation tasks (e.g., standardization of the date-time format or string manipulation). Data is transformed into an internal format by specific pre-processing modules before being used in the later steps of the analysis. Transformed logs are hosted in a staging database to ease the analysis. The modular organization of the analysis software enables redefinition of platform-dependent scripts in order to facilitate the analysis of different systems.

```

1. merge_all_logs();
2. sort_timestamps();
3. foreach event in the merged log {
4.   if (Time(next)-Time(curr)<W)
5.     Tuple->add(next);
6.   else
7.     start_new_tuple(next);
8.   curr=next;
9. }

```

Figure 1: Pseudo-code of the coalescence algorithm.

Step 2: Data coalescence. Failure events that happen at around the same time may correspond to different manifestations of the same fault/error in the system, e.g., different error detectors might be triggered, resulting in multiple logged events, or the same cause might generate different failures in close proximity. Time coalescence is based on the assumption that logged events that have the same cause will be close to each other in time. Figure 1 shows the procedure used in this step to coalesce the data. All logs (Platform OS, Application, Exception, and DB) are merged together and ordered by their timestamps. Then, all entries that have the same error type and happen within the same given time window are grouped in a single *tuple*: i.e., a collection of a number of events (failure entries) that happen near each other in time are considered to be related to the same cause. The duration of the tuple indicates the amount of time that passed between the first and last failure entries grouped together. Our toolset can estimate a suitable coalescence time window of 820 s (W in the coalescence algorithm in Figure 1) using a heuristic similar to one proposed in [6].

Step 3: Failure Analysis. Coalesced data are used to classify the failure modes of the platform under study and acquire a set of measures such as MTBF and failure rate. Finally, statistical inference tests are applied to the dataset to identify specific models and potential trends in the data.

4. FAILURE CHARACTERIZATION

In the considered period of 283 days, the system received 275,290 files from customers. The vast majority (86.13%) of the files were processed successfully. In 37,822 cases (13.25%) the files generated a validation failure during the initial validation stages and hence were not processed. In 1,475 other cases (0.62%) the processing of the customers' files failed because of platform failures, the subject of this study. In particular, the results of the coalescence process show that:

- 73.1% of all failures belong to only 3 failure types (**finding (4)**) out of 42 in the logs.

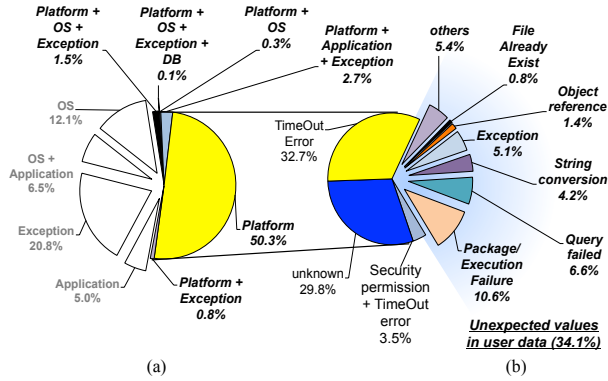


Figure 2: Distribution of the (a) tuple type after the coalescence process, (b) failure types (explosion of platform failures in (a)).

Table 3: Summary of the coalescence process.

Entries (raw data)		Tuples (after the coalescence)		
67285		1639		
platform failures	OS, DB, application, exception (no platform failures)	with platform failures		no platform failures
		918 (56.1%)		721 (43.9%)
		1 entry	> 1 entry	
1475	65810	679 (74.1%)	239 (26.9%)	

- Customers' data are responsible for 34.1% of the platform failures (**finding (1)**). Most of them are caused by unexpected values in customer data that escape the initial validation stages of the platform; such problems are difficult to foresee during the coding phase. Results show that 10.6% of these errors can be mitigated by performing intensive robustness testing or fault-injection-based studies [20].
- 32.7% of the platform failures are due to timeout errors (**finding (2)**), which tend to cascade. For the analyzed platform, elimination of timeout errors would increase the MTBF by a factor of 1.94, e.g., from 7h 2min to 13h 41min.

4.1 Output of the Coalescence Process

Figure 2(a) shows the outcome of the coalescence process of all data logs. Recall that the coalescence process aims to group together failure events that happen near each other in time (within the assumed time window of 820s). Therefore, the number of tuples obtained after the coalescence is lower than the number of entries contained in the raw data (i.e., before the coalescence). The coalesced data, summarized in Table 3, consist of 1,639 tuples. 1,475 platform failures are coalesced in 918 tuples and constitute 56.1% of the total tuples. 671 (73.1%) of the 918 tuples contain only a single platform failure. 239 (26.9%) of the 918 tuples with platform failures have more than one failure event. 195 of them (82%) contain distinct failures occurred in the same execution stage (see Section 2). The remaining 721 (44.9%) of 1639 tuples contain only entries from the OS, DB, application, and exception logs since they happened farther than the coalescence time window of 820s from a platform failure. Hence, such entries will not be considered in the analysis.

OS errors are not common. The OS logs contain 37,481 errors and warnings, but no OS failure events, such

as OS crash or hang, are observed. OS errors may impact platform failures when they occur in close proximity to each other. Such events reflect situations in which OS errors affect multiple computation stages of the platform, contributing to their failures. Only 88 tuples (5.4%) include one or more platform failures occurring in close proximity with one or more error in OS, DB, application, and/or exception logs. In 94% of such cases, the platform fails because of timeout errors. Corresponding tuples have an average duration of 697s, 5.7 times longer if compared to the average duration of 113s for the tuples with only platform failures.

The MTBF of 7h and 2min is estimated as the mean time between the 918 tuples grouping all the 1,475 platform failures. The MTBF computed as the mean time between the 1,475 platform failure entries gives a value of 4h and 22min, showing that, in average, 1.62 failures are grouped into a unique tuple by the coalescence process and most likely are due to a unique or cascading causes.

Logs contain 42 different types of failures. Figure 2(b) shows a breakdown of the failure types of all the tuples with platform failures. In 29.8% of the failures, logs fail to encapsulate meaningful messages that can be used to diagnose or classify the failure. Such entries consist mainly in empty strings and are classified as *unknown*. Timeout errors (32.7%), package/execution failures (failure in invoking/executing of external packages; 10.6%), and unknown (29.8%) categories account for 73.1% of all platform failures.

Timeout errors are a predominant source of failures. They are a known problem in the world of Microsoft SQL and .NET applications¹ when large quantity of data are being dealt with². In SQL Server, these errors appear to be due to i) performance, ii) concurrency, and iii) network problems. Business-oriented systems like the platform under analysis are characterized by small numbers of big transactions. These transactions are frequently INSERT and JOIN operations. Due to the unpredictability of data, time out errors can be caused by a transaction taking a long time (e.g. variable length data inserted into a large table), forcing other queries on the same table to wait, and potentially violate the timeout.

Another factor that may cause time out error is the level of concurrency in the system. Typically, .NET applications create a query by instantiating a specific SQLClient thread from a pool of available threads. If the thread runs out of SqlInternalConnection objects, it is placed on a waiting list. If the load and/or the level of concurrent queries are high, then the wait time can be bigger than the programmed timeout, causing timeout errors.

Network problems also play an important role in timeout errors. The platform uses UDP packets to connect to the SQL Server to obtain the connection endpoint (TCP port number/named pipe), and packets can be dropped and never retransmitted. That is likely to happen in a virtualized environment subjected to high CPU or I/O load, which may cause transient failures of the virtual network interfaces³.

An example of failures due to timeout errors in the co-

¹an example is reported in <http://technet.microsoft.com/en-us/library/cc966413.aspx>

²https://www.google.com/search?hl=en&as_q=net+timeout+error

³http://kb.vmware.com/selfservice/microsites/search.do?language=en_US&cmd=displayKC&externalId=1004087

Table 4: Example of a complex failure as reported after the coalescence (sanitized of confidential information). Entries from platform failures are reported in bold.

Tuple Details	Messages from the logs
Time Stamp: 2012/05/23 12:16:06 Duration[s]: 1594 Entries: 9 Type(num): OS(5) PLATFORM(4) Failure Type: Service/Driver failure + Package/Execution + Timeout Error	ERR{SCM->A timeout was reached while waiting for a transaction response from the LanmanServer service.) ERR{DCOM->The application-specific permission settings do not grant Local Activation permission for the COM Server application [...] ERR{SCMr->A timeout was reached [...] (30000 Netman)} ERR{SCM->A timeout was reached [...] (30000 UmRdpService)} P4_STG(FILE_ID->Platform(Timeout expired. The timeout period elapsed prior to completion of the operation or the server is not responding.)) WARN{RDPDR->An Io Request to the device DEVICE did not complete or canceled within the specific timeout [...] } P4_STG(FILE_ID->Platform(Timeout expired. The timeout period elapsed [...])) P4_STG(FILE_ID->Platform(Timeout expired. The timeout period elapsed [...]))

alesced data is shown in Table 4. It includes 5 OS errors and 4 platform failures grouped in the same tuple. The OS errors are due to services failing to (re)start, including some critical services like the *LanmanServer* (core service for the management of LAN services) and other services depending on it. Eventually, such services impact on the platform in the P4.STG component (upload of user files to the DB server accessed via LAN services). From the example it is clear that failures due to timeout may cascade and cause the failure of several other components downstream.

4.2 Key Factors Impacting Platform Failures

Unexpected values in customer data cause 34.1% of platform failures. Based on the content uploaded by the user (e.g., inventory data or sales data), the platform selects the required processing. The platform extracts data for the processing from the received files and feeds the data to downstream stages that handle data manipulation and transformation (see Section 2). For that reason, the platform enforces a strict validation of customer data once they are uploaded. Because of the sheer volume of data and the high volume of customers, unexpected values might be present and elude the initial validation checks. A simple example is a field containing an unexpected null or empty value that is then uploaded to a SQL table that does not accept the null value, or used as symbolic reference to an object causing a reference to a null pointer. Another example is an overflow or division by zero caused by an unexpected numerical value in the data. Our analysis indicates that such failures lead to 34.1% of the total platform failures. In particular, data-induced errors consist in i) string conversion failures (4.2%), ii) problems in extracting table/column (6.6%) and file (0.8%) names in the parsed data, iii) references to null objects (1.4%), iv) data out of range and arithmetic exceptions (5.1%), v) failure in the invocation/execution of external packages due to wrong parameters/value ranges (10.6%), and vi) other data-related exceptions (5.4%).

Unhandled software exceptions are generated in 5.1% of platform failures. Exception handlers embedded in the platform code are effective in capturing runtime exceptions 94.9% of the time. A total of 922 different software exceptions are present in the data. As illustrated in Figure 2(a), they are coalesced in 25.9% of the 1,639 total tuples (sum of the percentages of the tuple types that include exceptions). On average, 2.2 different exceptions are present in each tuple, with a peak of 29 in one case, showing that they tend to occur in rapid succession, namely within the assumed coalescence time window. The high effectiveness of exception-handling mechanisms is in part due to the strong support provided by the used programming framework (.NET) that enforces the use of try-catch blocks when, for instance, in-

voking string/array functions and accessing communication channels. This finding indicates that exception-handling code can be improved based on understanding of the data errors that lead to platform failures. Such code improvement would in turn substantially reduce the impact of errors such as string conversion problems, reference to null objects and arithmetic exceptions. In addition, understanding of unexpected data could also be used to design thorough robustness-testing campaigns in order to mitigate failures due to invocation of library functions with incorrect parameters/value ranges (10.6%).

Table 5: % of LOC count and failure density (number of failures / LOC) for the modules that exhibit 95% of platform failures. P3_PP is the only multi-threaded module.

STAGE	%LOC	failures	failure density
P3.P	14.39%	35	0.013
P3_PP (6 threads)	0.72%	120	0.89 (0.148/thread)
P4_PREP	2.54%	69	0.15
P4_STG	2.56%	179	0.37
P4_TRNF	2.74%	215	0.41
P5	3.75%	178	0.26
P6	0.35%	9	0.14
O4	11.11%	604	0.32

Five platform stages are responsible for 95% of platform failures (finding (4)). Recall that the platform executes in fixed processing steps (see Section 2), and all the components are subjected to the same load (they are used in sequence). Table 5 shows the breakdown of the platform failures per processing stage, relating the number of component failures to the size of the component expressed in lines of code (LOC). For each component we show the *failure density* expressed as the total number of failures divided by the the component LOC. Interestingly P3_PP, shows a higher failure density than the other components. P3_PP is the only multithreaded component executing 6 concurrent threads. The failure density of P3_PP, normalized with respect to the number of threads, is similar to the other data preparation stages (e.g., P4_PREP, P6), which suggests the presence of a persistent (across different components) failure cause.

Finally, all components in charge of data manipulation, transformation, and archiving (P4_STG, P4_TRNF, P5, and O4) show higher failure density than the others. In particular P4_TRNF and P4_STG include a unique COTS library for data manipulation, written in .NET, which is 7 times the size of the biggest component in the platform⁴. The

⁴Which is not included in the LOC count of Table 5. The absolute LOC count for the platform components is not disclosed for confidentiality.

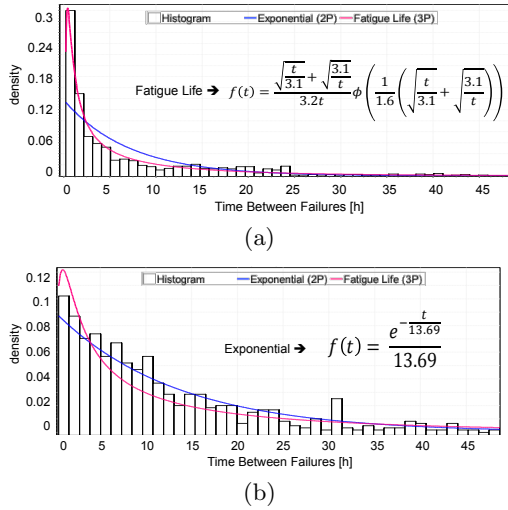


Figure 3: TBF density function distribution for (a) all the failure types, (b) all failure types but timeout failures.

library contributes to a substantial number of platform failures classified as either package/execution failures (10.6% of the total failures) or string conversion (4.2%), shown in Figure 2(b). Such failures are mainly caused by bad/corrupted values passed as parameters of the library functions, which are not robust to unexpected data values.

Another component showing high failure density is O4. When creating the archives, the component queries the database to create archives of the processed data. Failures exhibited by this component are timeout errors (47%), unknown (23%), simultaneous occurrence of failures due to wrong security permissions (for instance to move/create files in specific folders) and time out errors (5%) and others. While we cannot diagnose the failures classified as unknown, we can note that components that deal with querying the database, such as O4 are more prone to timeout errors. In summary, i) P3_P, P4_L2, P4_STG, P4_TRNF, P5, and O4 account for 95% of the total failures; ii) data manipulation and transformation components are more sensitive to unexpected/corrupted values in the customer data; iii) many failures are due to external components used for standard data manipulation and transformation tasks, such as data parsing and harmonization; and iv) components in charge of querying the database to archive the data are more susceptible to failures due to wrong security permissions and/or timeout errors (addressed in the next section).

4.3 Impact of Timeout Errors

Figure 3(a) illustrates the distribution of time between failures for the coalesced data. The distribution shows that *about 30% of failures happen within 1h after prior failures (finding 5)*. After the first hour, the probability of a second failure in rapid succession drops sharply. Interestingly, the data can be successfully fitted with a Fatigue life distribution [15]. The Kolmogorov-Smirnov test show a p-value of 0.04, showing that, with a confidence of 96%, the null hypothesis is that the data follows a Fatigue Life distribution cannot be rejected. Other distributions, like Weibull, lognormal and exponential do not obtain a comparable good-

ness of fit. For instance, the p-value obtained for fitting the data an exponential distribution is 0.21.

Fatigue Life distributions have been extensively used in mechanical reliability to model the fatigue damage process in metals [15] subject to cyclic load above a critical threshold. Cracks (comparable to errors) accumulate and eventually generate a failure of the metal structure. Hence, in our case, the distribution of the observed TBF data is similar to one dominated by an underlying process of error accumulation triggered by stressful workload cycles.

In order to investigate potential error accumulation, we identify the failure types that dominate the observed distribution. The analysis consists of eliminating one failure type at a time and re-computing the distribution fitting and MTBF. The objective is to observe what type of failure dominates the TBF. We repeated this operation for other failure types and we found that the only case in which the Fatigue Life distribution fail in fitting the data is achieved after the elimination of timeout error from the dataset (p-value of 0.14). Interestingly, in that case, shown in Figure 2(b), the dataset can be successfully fitted (p-value of 0.07) with an exponential distribution with a rate of $1/13.69[h]$, i.e., in the case of no failures due to timeout errors, the MTBF goes from 7h 2 min to 13h 41min. In particular, for the whole dataset, the reliability drops to 90% in 48 min, while for the same system without timeout errors the same reliability level is reached after 91 min.

The reason for the exponential inter-arrival of failures is that elimination of timeout failures means that most of the remaining failures (34.1%) happen because unexpected values in the customers' data. Since customers submit files independently, we are likely to observe independent failure inter-arrivals, as illustrated in Figure 3(b). Conversely, a failure in the processing of a file due to timeout error often introduces a further delay in the processing pipeline, eventually causing cascading failures in the other files being processed in the same or earlier stages. Elimination of the timeout failures (which happen with 94% of the OS errors and have a longer duration than other failures) would substantially reduce the risk of failures occurring in rapid succession because of error accumulation (they happen independently) and increase the reliability by a factor of 1.94.

5. FAILURE RATE ANALYSIS

In this section, we analyze the daily frequency of platform failures and its relationship with the workload. The analysis of the failure rate shows that:

- Increase in the failure rate depends mainly on increase in the workload intensity (i.e., rate of the files submitted per day) (**finding 7**), rather than on introduction of defects in the new version of the platform software (**finding 6**).
- There is no statistical evidence that workload volume (i.e., size of the files submitted for processing) impact the platform failure rate (**finding 8**).

5.1 Trend Analysis

Basic statistics on platform failure rate. Table 6 shows the month-by-month evolution of the key platform failure statistics (e.g., tuple duration or MTBF). During the observed 283 days, the platform underwent several updates

Table 6: Breakdown of synthetic statistics per month. The last month includes only a few days of data.

Month	num Failures	Tuples with a single failure entry	average duration tuples[s]	MTBF [h]	Median TBF [h]	avg. entries per tuple
1	106	96 (90.6%)	37	6.80	3.81	1.1
2	67	53 (79.1%)	103	10.20	4.9	1.3
3	85	73 (85.9%)	123	8.74	4.68	1.8
4	92	71 (77.2%)	135	7.83	2.73	1.9
5	143	115 (80.4%)	125	5.26	2	1.40
6	132	105 (79.5%)	108	5.42	1.78	1.4
7	104	71 (68.3%)	105	7.06	2.69	1.5
8	96	63 (65.6%)	253	7.77	1.94	2.0
9	109	44 (40.4%)	376	6.76	2.68	2.2
10	25	5 (20.0%)	484	8.00	4.23	2.5

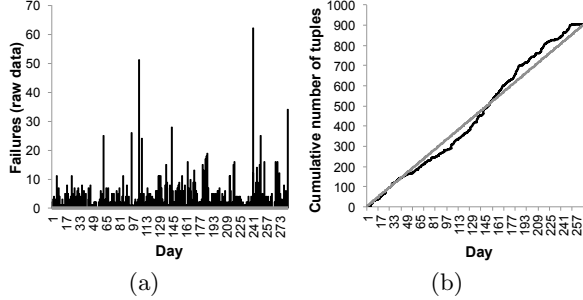


Figure 4: (a) Number of failures per day; (b) cumulative number of failures per day (line between the first and last value added to show an ideal constant pattern).

in order to meet newly on boarded customers’ demands in terms of amount and type of files to process. While a clear trend in the decrease/increase of the MTBF over the 10 months is not observed, we can see a substantial increase in the average number of failures coalesced in a single tuple and in the tuple duration. The average failure duration increases from 37s in the first month to 484s in the last month while in the first month 90.6% of tuples contain just a single failure entry against a 44% in the last month.

Figure 4 shows the temporal distribution of the number of platform failure entries per day. Only 25 days out of 283 pass without platform failures. Specifically, it can be observed that:

- Days 1 to 96: The platform (version v.3.1 of the software) exhibits 324 failures in 3 months (21% of the total failures, with a failure rate 3.4 failures/day).
- Day 97: The LATAM version of the software is deployed with new features for meeting the requirements of newly on boarded customers from the Latin American Market. This update introduces two new software components and modifies a subset of core modules (e.g., the data verification and data manipulation components of the platform; see Section 2) to handle the new type of workload.
- Days 98 to 187: The platform exhibits 630 failures (43% of total failures) in 3 months (failure rate of 7 failures/day, twice that of the first 3 months).
- Days 187 to 241: The platform exhibits 200 failures (13% of total failures) in two months (failure rate of 3.7 failures/day, comparable with the first 3 months).

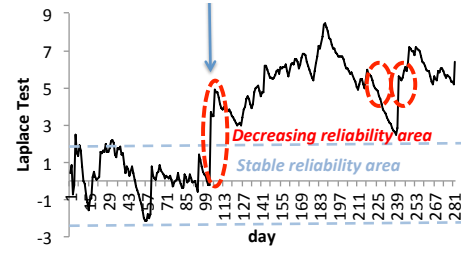


Figure 5: Laplace trend test.

Figure 4(b) shows the cumulative number of platform failures per day. A constant daily failure rate would have resulted in a linear increase in the cumulative number of failures, as illustrated by the straight line in Figure 4(b).

Trend Analysis. In our analysis we employ the Laplace trend test [18]. The objective of the Laplace trend test is to determine whether the arrival of failures significantly changes with time. It tests a null hypothesis that the system failure arrival process is a renewal process, i.e., an arrival process in which the inter-arrival times are independent and identically distributed. If this hypothesis can be rejected at some appropriate significance level, then it can be concluded that some changes in the reliability of the system are occurring.

A score greater than zero means that there is an upward or increasing trend and a score less than zero means there is a downward or decreasing trend. When the score is greater than (less than) $+1.96$ (-1.96), we are at least 95% confident that there is a significant trend upward (downward). That implies that successive inter-arrival failure times tend to become larger (smaller) for an improving (deteriorating) system. A score of zero means that the trend is a horizontal line. Figure 5 shows the result of the Laplace test. In particular:

- There is no substantial change in the reliability of the platform until the deployment of the LATAM version of the software on day 97.
- The deterioration of the platform reliability (an increase in the platform failure rate), indicated by the sharp increase (beyond the 1.96 mark) in the Laplace test results, happens just after the platform update/new clients from Latin America join the platform.
- The temporary reversion to the older version of the software on day 213 changes the trend until day 241, when the LATAM version of the platform software is restored again.

A decrease in the reliability may not, and usually does not, mean that the software has more bugs. The higher failure rate may be due to changes in the operational profile, e.g., software upgrades, type and/or an increase in the workload processed by the system (in our case, due new clients from the Latin American markets). The impact of those two factors on platform failures is discussed in the following.

5.2 Impact of Platform Software Upgrades

The platform upgrade on day 97 made possible to process new files from the Latin American market, increasing the number of served customers. Recall that the workload consists of files submitted to the platform for processing. Figure 6 shows the workload in terms of *intensity* (files submitted to the platform per day, regardless of the file size) and *vol-*

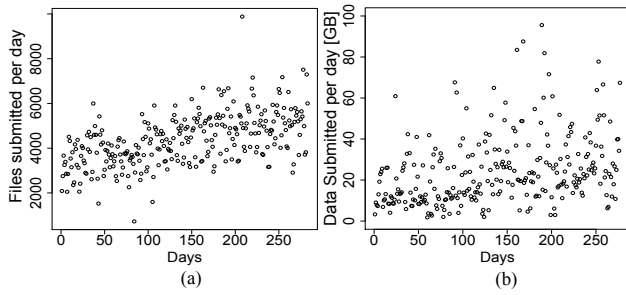


Figure 6: Workload: (a) number of files per day (*intensity*), (b) data submitted per day (*volume*).

ume (amount of data submitted per day, regardless of the number of files submitted for processing). Note that i) the higher the number of files, the higher the level of concurrency in the platform, and ii) the larger the files, the longer the processing time for each component of the platform. Figure 6 shows that the number of submitted files per day increases by a factor of 2.5 towards the end of the period because of an increasing number of served customers.

In order to obtain a distribution of the failure rate in which all the failures are equally represented regardless of the potential interaction with the workload, we i) normalize the daily failure rate and workload distribution by dividing the daily failure rate by the daily workload rate in terms of intensity (i.e., number of files submitted per day) and volume (GB of data submitted per day). We refer to those obtained failure rate figures as *absolute failure rate*, i.e., the failure rate per processed file or per processed GB of data. Figure 7 shows the cumulative absolute failure rate computed with respect to the workload intensity. Interestingly, the distribution shows a linear behavior for the observed period, except for a jump around the day of the platform software upgrade (day 97). The introduction (elimination) of software defects with the software upgrade would eventually cause the curve to have an increasing (decreasing) slope (dashed curves in Figure 7). The plot of the empirical data shows that the software upgrade did not introduce additional defects or reduce the slope of the graph. The jump around day 97 corresponds to a burst of transient failures manifested after the new clients joined the platforms. Similar results are obtained for absolute failure rate computed with respect to the workload volume, not shown here because of space limitations. A detailed analysis of the failure causes per customer, in day 97, showed that 79% of the failures are because of unexpected values in data from few new customers, i.e., about the double of the average 34.1% computed on the whole period (finding (1)). In particular, after a careful discussion with the development team, we identify that the learning curve of the platform and the configuration of the client software may cause higher failure rate in new customer, during their first weeks. Eventually, that showed to impact on the overall platform reliability. Hence, we can conclude that, (i) with the exception of a few days (e.g., day 97 and day 241; see Figure 4) the platform failure rate is substantially immune to the changes introduced by the platform software upgrade, while (ii) it is affected by the inexperience of new arrived customers or their client misconfiguration.

5.3 Impact of the Workload Volume

Let us consider the typical workload week (Monday to

Table 7: Analysis of the workload and failures by weekdays.

	Mon	Tue	Wed	Thu	Fri	Sat	Sun
n. failures	239	248	213	310	233	131	101
% failures/day	16.2%	16.8%	14.4%	21%	15.8%	8.9%	6.9%
% received files	16.1%	15.2%	14.4%	16.6%	14.6%	11%	11.9%
data [GB]	970	800	780	930	1390	1200	790
avg file size [MB]	24	20	21	22	37	42	78
failures/GB of data	2.44	3.02	2.73	3.36	1.68	1.08	1.29
failures/(n. files per day)	0.58%	0.64%	0.58%	0.73%	0.62%	0.46%	0.33%

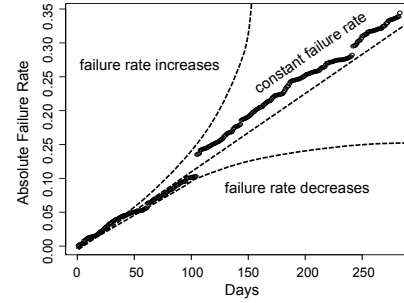


Figure 7: Cumulative absolute failure rate obtained by dividing the daily failure rate by the workload intensity (number of files processed per day).

Sunday) and relate the failure rate to it. Table 7 reports the breakdown of the workload and failure data per days of the week. As one might expect, the number of files submitted over the weekend is substantially lower (48% - 66% lower than for the other days). However, by the end of the week, customers submit larger files to compute (mostly reports on the week's sales), which can reach 2 to 3 times the average size of the files submitted earlier the week. As shown in Table 7, the ratio of failures to GB of data varies slightly for the first four days of the week and drops by a factor of 1.5 to 3 for the remaining three days⁵. In fact, during the weekend, the platform processes fewer files, but larger in size than the other days and we can conclude that the platform can tolerate a high volume of data without being affected.

5.4 Impact of the Workload Intensity

During the workdays, customers submit daily or hourly reports on their inventory or sales, smaller in size, but more frequent than the weekly reports sent over the weekend. Recall that the platform is organized in computation stages that work in a pipeline. Therefore, a higher intensity of the workload forces a higher level of concurrency in the system, as more threads (sub stages) are active in parallel to handle the amount of files to process. On days with higher concurrency (Mon. and Thu.) the percentage of failed files is higher than the other days, while, for instance, on the Sundays, the percentage drops to about 50% of the other days.

In order to confirm that a change in the workload causes a change in the failure rate we consider the Mann-Kendall statistical trend test. If we plot the workload intensity over the failure rate, this test evaluates the null hypothesis that the failure rate data have no trend over the workload data,

⁵ An analysis of variance, which is not included in this paper because of space limitation, confirms the hypothesis.

as illustrated in Figure 8. The result of the test reports a τ value of 0.485 (correlation) and $p - value = 2.22E - 9$, which suggests that the null hypothesis can be rejected, i.e., that there is a trend in the data. Therefore, we can conclude that i) the increment in the platform failure rate is due to the increment in the intensity of the workload (see Figure 6.(a)), and ii) the level of concurrency enforced in the platform by intensive workloads leads to an increase in the failure rate. In other words, the reliability of the platform remains constant over time (with respect to the absolute failure rate), while the usage of the platform grows in intensity. Therefore, the number of failures grows as well (**finding (9)**).

6. RELATED WORK

Analysis of operational failures in computer systems has been the focus of active research for quite some time. The focus of such work is on categorizing the nature of failures [5, 9, 17], identifying bottlenecks [14, 21] and bugs [12, 13, 19] and measuring the reliability of the system being analyzed [2, 3, 8, 16]. However, despite such efforts, there continues to be a dearth of empirical work on the failure analysis of commercial software systems. Our work is one of the few efforts to characterize the failures of an SaaS platform for the processing of a large amount of business data.

Several failure studies [4, 9] show that most of the failures experienced in enterprise software systems are caused by recurrent faults in a small proportion of the total code base. The authors of [9] demonstrate with empirical data that the 80-20 Pareto rule for operational failures (20% of the modules are responsible for 80% of the failures) holds in operational enterprise software. Our work shows that 95% of failures occur in 31% of the modules (5 out of 16) and that about 73% of the failures belong to 3 fault types out of 42. In addition, differently from traditional software, our SaaS platform benefits of a virtual environment that, as discussed in Section 4.1, can affect the failure modes of the platform.

There is little work that studies the relationship between workload and failures in data-processing software platforms. In [17] the authors study failures of SCOPE, a general-purpose framework used at Microsoft Bing to mine large datasets. They show that the majority of failures (84.5%) are due to unexpected values in the processed data. Our data show that unexpected values in user data cause 34.1% of the total failures. We argue that the smaller percentage is partially due to the strict data validation checks enforced by the platform in the first stages of the computation (see Section 2). If so, it suggests that in data-processing systems, data validation stages may be highly beneficial in reducing the impact of unexpected values on platform fail-

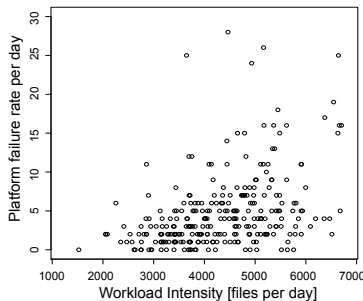


Figure 8: Failure rate vs. workload per day.

ures. We show that in business data-processing platforms like that considered in this study, high rate of user transactions (workload intensity) can cause substantial increment in the platform failure rate due to data-induced errors and timeouts. Our finding demonstrates the value of creating data-driven models to be used at runtime in cloud computing environment to decide whether to scale the system capacity up or down in order to reduce the impact the workload has on the platform failure rate [11].

7. LESSONS LEARNED

While this study focuses on a specific SaaS platform, we believe that there are lessons to be learned as discussed in the following. Results show that an in-depth analysis of failure data is the key to understanding factors that impact platform failures and suggest effective fixes. Table 8 summarizes the major changes in the platform and operational procedures introduced by the development team, based on the findings from this study (see Table 1).

Improving Data Validation Procedures. Most of the failures due to unexpected values in user data can be fixed without radical changes in the platform code, flow or algorithms. Usually, fixes consist in a few extra LOC in a limited number of critical components, such as those identified to manifest most of the failures (finding (4)). For example, 78 lines of exception handling code were added to the platform codebase to address the majority of the data-induced failures in the analyzed failure data. However, the heterogeneity of customers' data and their increasing number makes it hard to design comprehensive data validation procedures. Hence, the platform development team created a tool based on the analysis procedures described in this paper to assist in identifying assertions based on the failure data in order to improve the testing and the creation of exception handlers.

Fault Avoidance by Customer Training. New customers show a higher failure rate due to unexpected values in their data. This can be due to the learning curve of the platform as well as in configuration problems of their client software. The example is the decrease of the platform reliability after day 97 because of the newly added clients from the Latin American market (See Figure 5). A strategy enforced by the SaaS management team to mitigate such failures consists in profiling customers for type and frequency of failures to suggest whether they should be (re)trained or have their client software reconfigured/upgraded. In few months, this fix contributed to solve 20-40% of the failures per customer, for new customers.

Improving the Resiliency to Intense Workload. Intensive workloads stress the platform, increasing its susceptibility to failures for two main reasons: (i) a higher chance of receiving data with unexpected values because of high number of files submitted per day, and (ii) load induced software failures, namely failures triggered under high level of concurrency (in our case, 32.7% of all the failures - see Section 4). While (i) can be addressed with better exception handlers, (ii) requires a fine tuning of the platform, namely, the timeout values and the number of available virtual machines. This way, without changing the application code, the team was able to fix about 85% of timeouts. From our analysis (see Section 4.3), this would correspond to a MTBF increased from 7h and 2 minutes to about 12h.

Improving the Time To Recover. 29.8% of the analyzed failures generate empty log entries. This translates

Table 8: Major platform improvements based on the discussion of the findings with SaaS platform developers

Findings	Problem Causes	Fix	Fix Effectiveness
About 34% of the platform failures are due to unexpected values in customers' data (Section 4.2)	Tradeoff between cost and comprehensive data verification: (a) thorough checking of each data entry is too costly, (b) hard to eliminate unexpected data values in batch file processing system. 2. Customers (from emerging markets) are frequently not IT savvy.	1. Improvement of Exception Handling; 2. Creation of a tool based on the analysis tool in this paper to extract specific assertions from the failure data. 1. Profiling customers for type and frequency of failures; 2. Customer (re)training, e.g., format of the allowed data; 3. Reconfiguration and/or upgrade of the client software.	Fixes to exception handlers ranges in 3-10 LOC of extra code per each exception handler. Testing and Incremental improvement of exception handling made easier. Reduction of failures due to bad data by 20-40. No modifications to the software
Nearly 33% of platform failures are due to timeout The failure rate increases if the workload intensity increases (Sections 4.3, 5.3)	With increasing number of clients, database becomes the bottleneck; timeout errors accumulate and can affect the processing of several consecutive files.	Increase the timeout value (from 60s to 120s) without violating the SLA Increase the number of virtual machines so to decrease the load on each of them by e.g., 20%	Reduce the timeout errors by 85% Projected increase of MTBF from about 7h to about 12h.
About 95% of platform failures can be attributed to a few (5 out of 18) processing stages (Section 4.2)	The identified stages are: (a) functionally more complex than others, (b) interact more frequently with the database	Identification of critical components: (a) use fault injection to pinpoint error sensitive functions (b) introduce application-specific assertions for runtime error detection	Fault/error injection can assist in diagnosing causes of critical failures, e.g., inconsistency of structural information in the database
29.8% of failures generate empty log and are classified as unknown (Section 4.1).	Inability of the platform to log errors.	Include specific logging features in the platform codebase.	Reduce unknown failures to 4% against adding extra LOC (100) to few critical components.

into a significant amount of failures that cannot be successfully diagnosed and recovered, impacting the overall platform availability, and, eventually, leading to Service Level Agreement (SLA) violations. To this aim, the development team included specific logging features in the platform codebase, adding about 100 extra LOC. Lab evaluation of such improvement was shown to be effective in reducing unknown failures to 4% of the total failures. The improvement does not impact the core functionalities of the platform and was able to reduce the time to diagnose a problem, therefore improving the availability. Logs are a rich, ubiquitous source of information on production systems, worthy of investment.

8. ACKNOWLEDGMENT

This work is partially supported by the NSF grants CNS 10- 18503 CISE, CNS 13-14891, and by a grant from Infosys Ltd., Bangalore. We thank Jenny Applequist and Carol Bosley for their help on the earlier version of this paper.

9. REFERENCES

- [1] G. M. S. Analysis. Customer relationship management software, worldwide. 18 april 2013 2012, report reference id:g00248529.
- [2] S. Banerjee, H. Srikanth, and B. Cukic. Log-based reliability analysis of software as a service (saas). In *Software Reliability Engineering (ISSRE), 2010 IEEE 21st International Symposium on*, pages 239–248, 2010.
- [3] A. Bondavalli, S. Chiaradonna, D. Cotroneo, and L. Romano. Effective fault treatment for improving the dependability of cots and legacy-based applications. *IEEE Trans. Dependable Secur. Comput.*, 1(4):223–237, Oct. 2004.
- [4] M. Brodie, S. Ma, L. Rachevsky, and J. Champlin. Automated problem determination using call-stack matching. *Journal of Network and Systems Management*, 13(2):219–237, 2005.
- [5] M. Cinque, D. Cotroneo, and S. Russo. Collecting and analyzing failure data of bluetooth personal area networks. In *Dependable Systems and Networks, 2006. DSN 2006. International Conference on*, pages 313–322, June 2006.
- [6] C. Di Martino. One size does not fit all: Clustering supercomputer failures using a multiple time window approach. In J. Kunkel, T. Ludwig, and H. Meuer, editors, *Supercomputing*, volume 7905 of *Lecture Notes in Computer Science*, pages 302–316. Springer Berlin Heidelberg, 2013.
- [7] C. Di Martino, M. Cinque, and D. Cotroneo. Assessing time coalescence techniques for the analysis of supercomputer logs. In *In Proc. of 42nd Annual IEEE/IFIP Int. Conf. on Dependable Systems and Networks (DSN), 2012*, pages 1–12, 2012.
- [8] C. Di Martino, M. Cinque, and D. Cotroneo. Automated generation of performance and dependability models for the assessment of wireless sensor networks. *Computers, IEEE Transactions on*, 61(6):870–884, June 2012.
- [9] N. Fenton and N. Ohlsson. Quantitative analysis of faults and failures in a complex software system. *IEEE Trans. on Software Engineering*, 26(8):797–814, 2000.
- [10] G. Forecast. Software as a service, worldwide, 2010 - 2015, www.gartner.com/it/page.jsp?id=1735214.
- [11] R. Ganesan, S. Sarkar, G. Goel, and C. Di Martino. Measurements-based analysis of workload-error relationship in a production saas cloud. In *2012 IEEE 23rd International Symposium on Software Reliability Engineering (ISSRE), Industry Track*, pages 96–105, 2012.
- [12] K. Glerum, K. Kinshumann, S. Greenberg, G. Aul, V. Orgovan, G. Nichols, D. Grant, G. Loihle, and G. Hunt. Debugging in the (very) large: ten years of implementation and experience. In *In Proc. of the ACM SIGOPS 22nd symposium on Operating systems principles, SOSP '09*, pages 103–116, New York, NY, USA, 2009. ACM.
- [13] P. J. Guo, T. Zimmermann, N. Nagappan, and B. Murphy. Characterizing and predicting which bugs get fixed: an empirical study of microsoft windows. In *In Proc. of the 32nd ACM/IEEE Int. Conf. on Software Engineering, ICSE '10*, pages 495–504, New York, NY, USA, 2010. ACM.
- [14] S. Han, Y. Dang, S. Ge, D. Zhang, and T. Xie. Performance debugging in the large via mining millions of stack traces. In *In Proc. of 34th Int. Conf. on Software Engineering (ICSE 2012)*, pages 145–155, 2012.
- [15] Y. Lee, M. Barkey, and H. Kang. *Metal Fatigue Analysis Handbook, 1st Edition*. Butterworth-Heinemann, 2002.
- [16] P. L. Li, R. Kivett, Z. Zhan, S. Jeon, N. Nagappan, B. Murphy, and A. J. Ko. Characterizing the differences between pre- and post-release versions of software. ICSE '13, pages 716–725, Los Alamitos, CA, USA, 2011. IEEE Computer Society.
- [17] S. Li, H. Zhou, H. Lin, T. Xiao, H. Lin, W. Lin, and T. Xie. A characteristic study on failures of production distributed data-parallel programs. In *In Proc. of the 2013 Int. Conf. on Software Engineering, ICSE '13*, pages 963–972, Piscataway, NJ, USA, 2013. IEEE Press.
- [18] M. R. Lyu, editor. *Handbook of software reliability engineering*. McGraw-Hill, Hightstown, NJ, USA, 1996.
- [19] S. Murtaza, N. Madhavji, M. Gittens, and Z. Li. Diagnosing new faults using mutants and prior faults (nier track). In *In Proc. of 33rd Int. Conf. on Software Engineering (ICSE) 2011*, pages 960–963, 2011.
- [20] R. Natella, D. Cotroneo, J. Duraes, and H. Madeira. On fault representativeness of software fault injection. *IEEE Trans. on Software Engineering*, 39(1):80–96, 2013.
- [21] Z. Yin, X. Ma, J. Zheng, Y. Zhou, L. N. Bairavasundaram, and S. Pasupathy. An empirical study on configuration errors in commercial and open source systems. In *In Proc. of the 23rd ACM Symposium on Operating Systems Principles (SOSP), SOSP '11*, pages 159–172, New York, NY, USA, 2011. ACM.